

THE UNIVERSITY *of* EDINBURGH
School of Physics
and Astronomy

Data Acquisition and Handling Project

Ultrasonic Transducer Driven Motion Detector

Ben Morrison
November 2025

Abstract

The aim of this project is to create a motion detector for a home security system using a HC-SR04 ultrasonic transducer and a Raspberry Pi. The HC-SR04 is capable of emitting and detecting ultrasonic pulses. Aided by a python script on the raspberry pi, the time between pulses can be measured and thus distance between sensor and object can be computed. This concept is extended to detect an obstruction between the detector and a reference object in front of it. The code should be easy to modify so as to allow flexibility in actions taken upon detection of movement. The project was completed in a four week period, consisting of ~ 12 hours spent inside the lab. The direction of the project was aided by the provided course booklet, [8].

Declaration: I declare that this project and report is my own work.

Signature: 

Date: November 26, 2025

Contents

1 Introduction **3**
- 1.1 Piezoelectricity . . . 3
- 1.2 Ultrasonic Transducer Module . . . 3
- 1.3 Raspberry Pi . . . 4
- 1.4 Limitations . . . 4

2 Calibration **4**
- 2.1 Range . . . 5
- 2.2 Effective Angle . . . 5
- 2.3 Effect of Surface Area . . . 6
- 2.4 Reading Intervals . . . 6
- 2.5 Speed of Sound . . . 6
- 2.6 Background sound . . . 7

3 Programming **7**
- 3.1 Important Methods . . . 7
- 3.2 HCSR04.monitor_old() . . . 8
  - 3.2.1 Pitfall . . . 8
- 3.3 HCSR04.monitor () . . . 9
- 3.4 Comparison . . . 10

4 Conclusion **10**

Bibliography **11**

A Supplementary Material **12**

1 Introduction

The main functional component in the ultrasonic transducer, henceforth denoted UTM, is the Piezoelectric crystal. The construction of the UTM can be seen in figure 1.

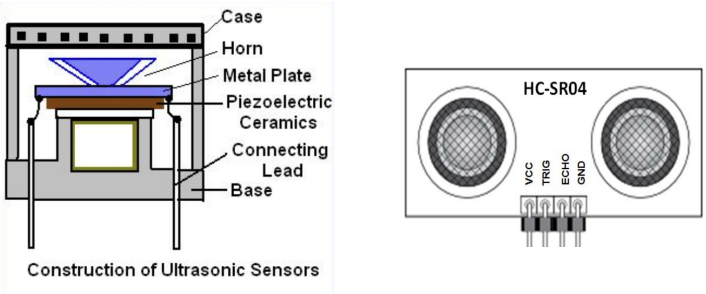


Figure 1: A side view of the construction of the UTM sensor (left) [7] and a front view of the HC-SR04 (right) [7]

The UTM uses a Piezoelectric crystal to create sound waves from an electric current and vice versa. To understand this, the model proposed by Bogdanov in 2022, [2], will be explained briefly. Technical details not relevant to this project, such as the specific crystal types which exhibit Piezoelectric behaviour, will be omitted from the explanation.

1.1 Piezoelectricity

When a certain type of crystal is subject to an electric field, the molecules are displaced slightly from their equilibrium. This shifts the charges around in the unit cell and a net polarisation is formed which propagates throughout the crystal. By alternating the electric field rapidly, the atoms will begin to oscillate in response. This mechanical oscillation propagates to nearby matter, such as air, creating sound and is known as the Reverse Piezoelectric effect. The direct Piezoelectric effect is the inverse of this process; by applying a mechanical stress to the crystal, the atoms are displaced slightly from equilibrium and a net polarisation is formed. This creates an electric field which drives a current in nearby conductors.

1.2 Ultrasonic Transducer Module

Now that the surface level mechanics of the UTM have been laid out, discussion will pivot towards its operation. The module has four pins: V_{cc}, TRIG, ECHO and GND (as seen on the right side of 1). The voltage supply and ground pins are self explanatory; these provide power and a ground connection to the module. The TRIG and ECHO pins are sensitive to only HIGH and LOW voltages so only boolean recordings are possible. A HIGH signal sent to the TRIG pin will tell the UTM to turn on a current across the metal plate, which rests flush with the Piezoelectric crystal in 1. Switching the current on for $1\mu s$ produces a mechanical oscillation of frequency $40kHz$, [7], in the crystal which in turn propagates to $40kHz$ ultrasonic waves in the air (this specific timing produces a packet of 8 ultrasonic waves). If the ultrasonic pulses make it back to the UTM, the air particles oscillating at $40kHz$ relay these oscillations back into the crystal, creating a mechanical disturbance, which induces a current in the metal plate. This current is amplified to a HIGH signal and switches the ECHO pin to HIGH. The device cannot provide information on the amplitude of the received ultrasonic pulses because the current induced in the metal plate is very small. This does not have an affect on the project as the detection of movement requires only the measurement of the total journey time.

1.3 Raspberry Pi

The python library RPi.GPIO, [3], is used to facilitate communication between the Raspberry Pi and the UTM. The circuit used is very simple and detailed in [2].

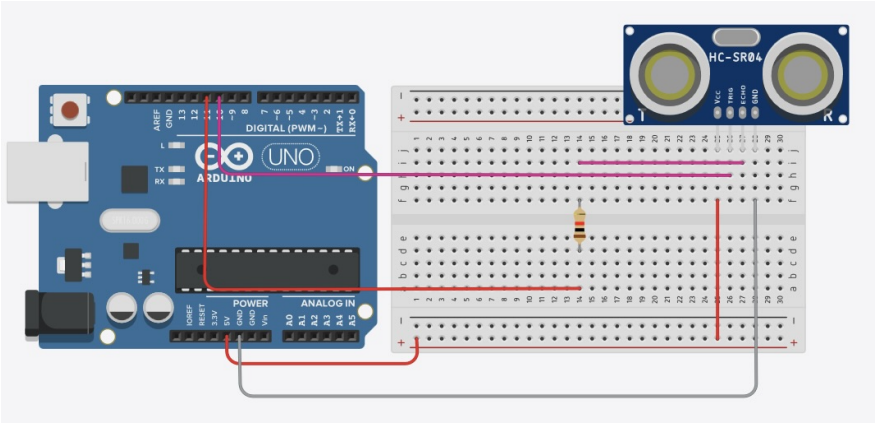


Figure 2: A top-down view of the UTM circuit. The figure was created using the Autodesk Tinkercad webapp, [1]. Note: the figure depicts an Arduino in place of a Raspberry Pi; the Tinkercad webapp does not have a Raspberry Pi component.

The only technical nuance worth mentioning with the Raspberry Pi is that the clock is out of sync. This means that attempts to create a log file in the lab have incorrect timestamps. In a real world scenario, care must be taken to ensure the Raspberry Pi's clock is synced whenever the device is restarted.

1.4 Limitations

It is stated in the UTM manual that the HC-SR04 model is only capable of accurate measurements in a 30° cone up to a distance of $3m$ away. The manual does not specify the plane or origin for this rotation. Calibration measurements taken in the lab suggest that the angular specifications are a modest underestimate, [2]. The measured effective angle was $30 \pm 0.5^\circ$, which corresponds to a 60° cone. The distance covered on the ground by this angle is determined using simple trigonometry:

$$\text{Ground distance} = \text{Height} * \tan(\theta) \tag{1}$$

where θ is the effective angle. Assuming the average ceiling is around $\sim 2.5m$ tall then the corresponding ground distance is $\sim 1.4m$. In the manual it is also stated that pulses should be separated by at least $60ms$; this was confirmed in the lab measurements [2]. This leaves a blind spot between pulses where an object could slip through undetected. An object must have a speed in excess of $23.3ms^{-1}$ to take advantage of this. This is not achievable for humans alone so the setup is appropriate for a home security system.

2 Calibration

Calibration measurements were taken to confirm/adjust the manufacturers claims. The claims are that the device provides accurate distance measurements up to $3m$ normal from the surface of the sensor with a 15° effective angle, [7]. The origin and plane of this angle are not made clear in the manual so both the vertical and horizontal were tested. The manual also claims that the area of the incident object must exceed $0.5m^2$ for accurate measurements — in practice this is a very large area and there were no easily movable objects in the lab which met this requirement.

2.1 Range

To calibrate the range, the sensor was fixed to a bench in the lab. A meter stick was used to determine the 'true' separation of the object and the sensor. To eliminate statistical noise, 30 readings were made per measurement with the mean value taken as the measurement. Between measurements the object was shifted by $0.2m$ (except for the smallest surface area object, which was incremented by 0.05m due to a fast deviation from the meter stick value). The meter stick had to be removed before taking the measurement as the sensor was consistently measuring the edge of the meter stick instead of the desired objects. The process was repeated for three objects with varying surface areas; $0.0177(1)m^2$, $0.0665(3)m^2$ and $0.2424(5)m^2$. The plots of the UTM measurement against the meter stick's measurement are in 3. The relationship is linear up until around $4m$ for the medium and large objects; the equation of the slope for the medium object is $y = 0.96x - 0.01$ and this deviates only slightly for the large and small objects. This is likely due to the fact that none of the three areas are close to the recommended surface area of $0.5m^2$. This data can be used to calibrate the UTM by inverting the linear relationship and rearranging for x (which is the meter stick reading). It should be noted that in the final version of the code, the line for this calibration is commented out because the UTM that was used in weeks 1–3 was faulty in week 4 and needed to be replaced. The remaining lab sessions afforded no time to repeat these measurements.

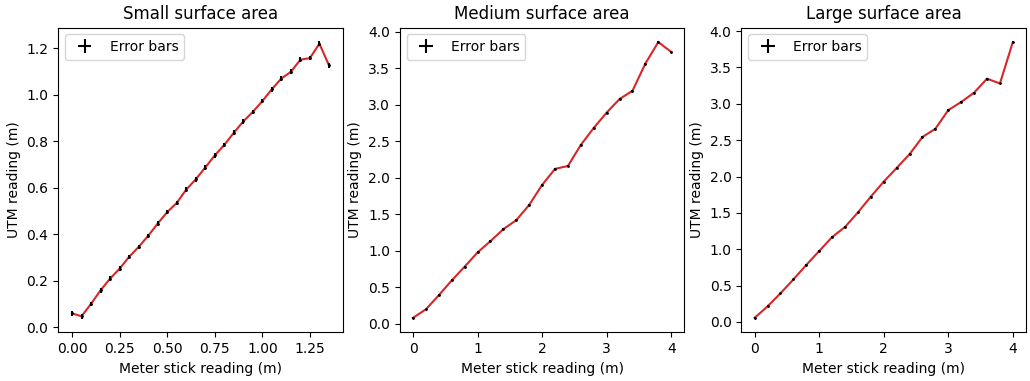


Figure 3: Plot of distance normal to the UTM sensor as measured by both the UTM and the meter stick.

2.2 Effective Angle

To calibrate the effective angle the UTM was, again, fixed to a lab bench. A protractor was attached in front of the sensor so that the $90°$ mark bisected the UTM. A meter stick was then placed so that the corner was touching the center of the protractor. The 'medium' surface area object was then appended to the end of the meter stick. The angle was increased in increments of $1°$ until the distance recorded by the UTM deviated significantly from the meter stick's length. At this point the angle was decreased by $0.5°$ to make use of the protractors $\pm 0.25°$ precision. A plot of the UTM reading vs θ, where θ is the angle between the $90°$ mark and the length of the meter stick protruding from the corner that is resting on the center of the protractor, is shown in 4.
It can be seen that the effective angle claimed in the manual, [7], is half of the measured effective angle — well within tolerance.

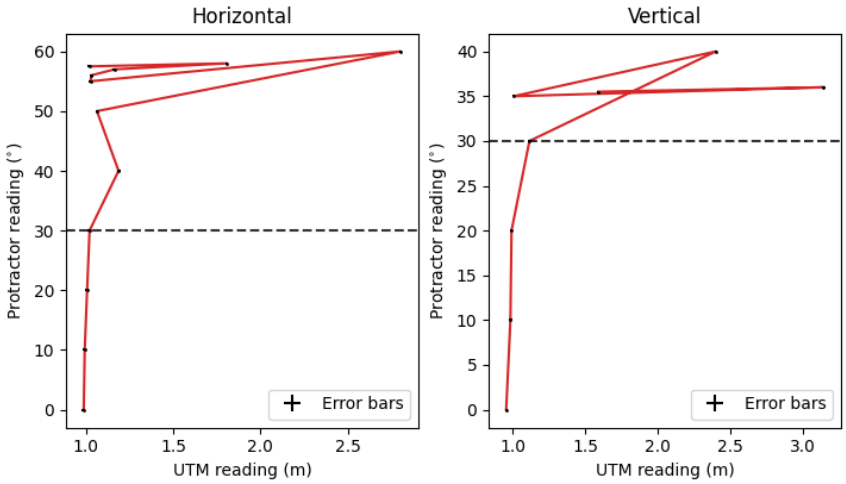


Figure 4: Plot of the length of a meter stick as measured by the UTM as a function of angle measured from the line normal to the midpoint of the Piezoelectric sensors.

2.3 Effect of Surface Area

The effect of surface area was not investigated extensively due to a lack of suitable objects in the lab. From the measurements made in 2.1, the readings were accurate up until $\sim 4m$. This suggests that the precision of the device was, again, underestimated by the manufacturer. For the remainder of the project only the large or medium surface area objects were used.

2.4 Reading Intervals

The fact that the manual recommends the readings be spaced out by $\sim 60ms$ was not realised until a significant amount of testing had already taken place. The consequence of this is described in detail in 3.2.1. These observations confirm that the reliability of the sensor is significantly degraded when the time interval between readings is too short. Assuming the device is in use for detection of human obstruction only, then the reading interval time can be calibrated so that the failed count threshold to trigger a detection lines up with the minimum time for a human to cross the ground distance (provided it is less than $60ms$). For example, the calculations in 1.4 can be reversed with a velocity of $v = 9.58ms^{-1}$, the speed Usian Bolt ran $100m$ at the 2009 Berlin World Champions, [4]. This will minimise the false detections due to insufficient spacing between readings whilst also safely overestimating the speed of a human obstruction.

2.5 Speed of Sound

The speed of sound is used to convert the timing between intervals into a distance. The direction of the closest object need not be normal to the sensor, so care was taken during testing to ensure that the closest object to the sensor (within the bounds of the effective angle cone) was the target object. In practice the distance measurement is not needed, but the results are easier to understand intuitively by converting the time readings into a distance. The expression used to do this was, [5]:

$$v_{sound} = 331ms^{-1}\sqrt{\frac{T}{273K}} \tag{2}$$

which is a function of temperature only. The room temperature in the lab was assumed to be $T = 300K$ for the duration of this project. The breadboard circuit could be modified to include a Dallas DS18B20 temperature sensor, [6], so the speed of sound can be more accurately calibrated to the climate of the room. An example of this circuit is shown in 5.

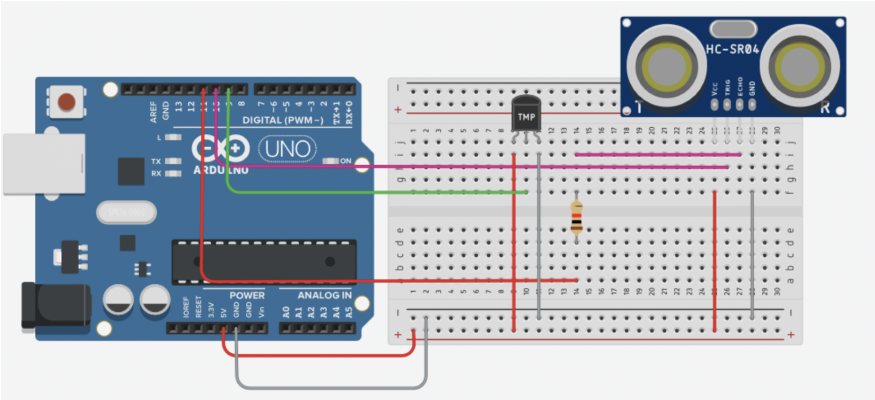


Figure 5: Circuit with both a HC-SR04 ultrasonic transducer and a Dallas DS18B20 temperature sensor connected to GPIO ports on an arduino. In practice, a Raspberry Pi would be used in place of the Arduino.

2.6 Background sound

The effect of background sounds on the accuracy of the UTM measurements was not investigated. This project could be extended in the future to incorporate controlled background sounds using the provided electronic speakers. If a substantial relationship with either the amplitude or the frequency of the background sounds is found then a microphone could be incorporated to adjust readings accordingly.

3 Programming

The main body of the code is a python class called `HCSR04` in the `sensor.py` file. To encourage modularity and (theoretically) reduce the time troubleshooting the code in the lab, an object oriented approach was taken. The actual file being run when the sensor is active is `detection.py`. Both of these files will be submitted along with the raw data (in the form of .ods files) and the code used to produce figures.

3.1 Important Methods

There are many methods in the HCSR04 class but only a few need to be discussed in detail. The `HCSR04.reading()` method is the only place in the code where the Raspberry Pi interfaces with the UTM. As the name suggests, it returns the distance that a single ultrasonic pulse travels on it's journey from sensor to object and back. The `HCSR04.calibrate()` method takes a large number of readings from a static target and computes important values such as the standard deviation of distance measurements, the average time taken for a pulse to return to the sensor and others which are detailed in the docstrings inside `sensor.py`. The results are saved graphically as .pdf files — an example of an output from `HCSR04.calibrate()` for the `HCSR04.monitor()` method is 6. The distance argument in `HCSR04.calibrate()` was a key component of the first attempt at detecting obstruction between the UTM and a reference object, however, in the final implementation the argument is not required. Upon detection of an event, the `HCSR04.detection_event()` method is called. The default behaviour of

this method is to append to a log file, indicating an obstruction has been detected. An example of a log file entry:

```
[2025-10-28 01:10:25.681764] DETECTION
```

The use case of this method is limited only by one's imagination — for example, an alarm could be sounded with the provided electronic speakers or an LED sequence initiated on the breadboard. There is also a `HCSR04.non_detection_event()` method which is called upon successful return of an ultrasonic pulse. By default this method does not do anything but it is possible to, for example, detect when an object *stops* moving. The `HCSR04.monitor()` and `HCSR04.monitor_old()` methods are the two object obstruction techniques attempted and will be discussed in detail in the following sections.

3.2 HCSR04.monitor_old()

Inside a home setting, it is not impossible that an object between the sensor and the floor could drastically reduce the effective range of the UTM (for example, if the sensor is placed on the ceiling, a ceiling lamp could obstruct the sensor's line of sight to the floor — in this case an intruder can easily avoid detection by passing underneath the ceiling lamp because the UTM will be calibrated to the ceiling lamp instead of the floor). This approach attempts to circumvent this vulnerability by sleeping the UTM for a time interval before listening for the ECHO signal in the `HCSR04.reading()` method. This means that the ultrasonic pulses returning from household objects which obstruct the sensor's line of sight will be ignored. The time interval to sleep the UTM can be determined by measuring the distance between the UTM and the desired reference object (i.e. the floor in the previous example) and feeding this information into the calibration process. By continuously taking measurements in this fashion the ultrasonic pulse from the reference object (theoretically) should be measurable by an ECHO pin HIGH signal every time. If this is the case, then using the average time for a call of `reading()`, the expected number of ECHO pin signals inside a predetermined time interval can be predicted and compared to the counted value:

$$\#\text{ of expected counts} = \frac{\text{time interval}}{\text{pulse time}} \tag{3}$$

A missing count suggests an obstruction between the sensor and the reference object. In practice, the variation in readings from the UTM create many missing counts so a threshold of 4 standard deviations is enforced. With this confidence interval, there is a 0.003% chance that a random deviation would lead to a missing count, as can be seen on a standard Z table, [9].
To further eliminate the chance of a false detection, kinematics of obstruction were considered. With the example of a $2.5m$ high ceiling, an object needs to travel at a speed of at least $23.4ms^{-1}$ to cross the $1.4m$ ground distance that the detector covers, 1.4. This means the detector is blind for $t = \frac{1.4}{23.4}\frac{m}{ms^{-1}} \approx 0.06s = 60ms$. Given that the average time for a reading is $\sim 1ms$ then an object obstructing the detector will cause at least 60 missed counts. Thus, a detection event is trigged only after 60 missed counts in a single time interval.

3.2.1 Pitfall

When testing this method in the lab the log file was plagued with false readings. Several hours were spent trying to narrow down the cause but the conclusion was that something was fundamentally wrong with the logic in the programming. The `HCSR04.monitor()` method was rewritten and a new method was tested. In hindsight, it appears that the root cause could have been the short time interval between calls of the `HCSR04.reading()` method. When attempting to debug the code it was noted that there were lots of readings $\sim 0.01m$ which pulled down the mean value used in `HCSR04.calibrate()`; this can

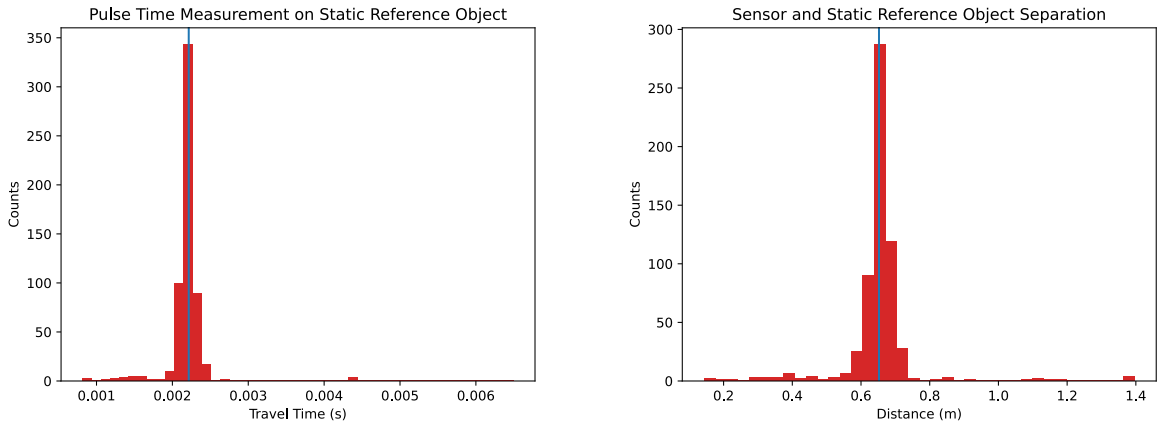


Figure 6: Histograms of the calibration readings generated by the calibrate method for the second approach of the monitor method, 3.3. The left figure shows the average amount of time taken for one call of the reading method to return a measurement of the distance between the sensor and the reference object. The right figure is the value of that measurement. They are both normally distributed with a sharp peak at the average value, represented by the blue line.

be seen in 7. This phenomena explains the manuals suggestion to allow at least $60ms$ between ultrasonic pulses to avoid the pulse from the TRIG pin directly triggering the ECHO pin. This could be circumvented by ignoring readings of the order $\sim 0.01m$ and adjusting the expected counts accordingly. Whilst this does theoretically solve the false reading problem, it would allow bypassing the detector by simply covering it. One could also solve the issue by sleeping the program for $60ms$ in the `HCSR04.reading()` method so the measurements are adequately spaced out.

3.3 HCSR04.monitor ()

This method was quickly brainstormed and programmed within the final hour of the fourth (and final) lab session. By measuring the distance of the (static) object closest to the sensor, small variations around an average value will be observed (exactly as discussed in 3.2). If the variations exceed those expected by a normal distribution (again with a confidence interval of four standard deviations), the object very likely has a non-zero velocity. This non-zero velocity is a sign of either 1.) an object entering the line of sight of the sensor or 2.) an object already in the sensor's line of sight being accelerated. Both of these actions suggest an intrusion so a detection event is triggered. Upon testing this method, a short burst of false detections is observed when `HCSR04.monitor()` is called. Ignoring these, the sensor was tested and successfully detected a ball rolling between the sensor and the large surface area object. A video of this process is included in the submission. In the video, the log file entries printed to the console indicate detection in real time. To remedy the false detections on startup, a warmup interval was introduced to the `HCSR04.monitor()` method. A for loop performs 100 `HCSR04.reading()` calls before expecting any return signals. The warmup period is clearly indicated in the log file to avoid confusion (this functionality was not tested in the lab as the lab session was concluding, however the implementation is relatively trivial). It should be noted that there is no way to test the code without the physical hardware.

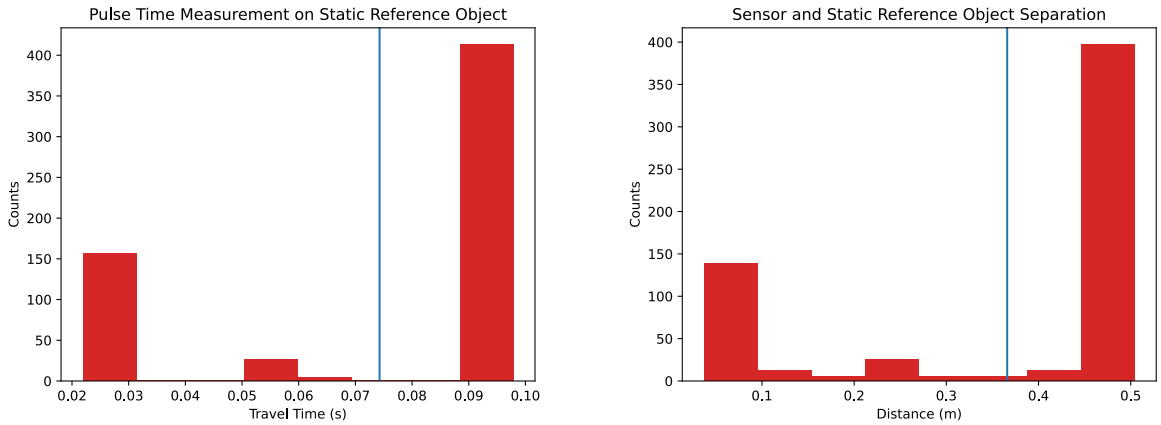


Figure 7: Histograms of the calibration readings generated by the calibrate method for the initial monitor method attempt, 3.2. It can be seen on both graphs that there is a significant amount of false readings which pull down the mean from its true value.

3.4 Comparison

Both methods theoretically deliver the desired functionality, however in practice only the `HCSR04.monitor()` method was observed successfully detecting motion in front of the sensor. The `HCSR04.monitor_old()` method is significantly more complicated than `HCSR04.monitor()` but, if it did work, would be the preferred method as it minimises the effect of interference from household objects (in turn making the system less prone to error).

4 Conclusion

To conclude, the accuracy claims of the manufacturer were confirmed for the HC-SR04 Ultrasonic Transducer Module. The UTM was controlled by a Raspberry Pi which runs a python script to detect movement in between the sensor and a reference object. The setup demonstrated reliable detection of movement in front of the sensor in a laboratory setting. The codebase for the module has been written in an object oriented style so that the functionality of the system, such as what actions to take upon detection of movement, can be changed easily without having to understand the underlying programming logic. With an additional lab session, the original implementation for the `HCSR04.monitor()` method could have been fixed, resulting in the system being less prone to faulty detections.

Bibliography

[1] Autodesk. *Tinkercad*. URL: https://www.tinkercad.com/ (visited on 11/22/2025).

[2] S.V. Bogdanov. "The origin of the piezoelectric effect in pyroelectric crystals". In: *IEEE Transactions on Ultrasonics, Ferroelectrics, and Frequency Control* 49.11 (2002), pp. 1469–1473. DOI: 10.1109/TUFFC.2002.1049727.

[3] Ben Croston. *RPi.GPIO 0.7.1*. URL: https://pypi.org/project/RPi.GPIO/ (visited on 11/22/2025).

[4] Olympic Games. *Bolt strikes in Beijing 2008*. URL: https://www.olympics.com/en/news/bolt-strikes-in-beijing-2008 (visited on 11/23/2025).

[5] OpenStax. *17.3: Speed of Sound*. URL: https://phys.libretexts.org/Bookshelves/University_Physics/University_Physics_(OpenStax)/Book%5C%3A_University_Physics_I_-_Mechanics_Sound_Oscillations_and_Waves_(OpenStax)/17%5C%3A_Sound/17.03%5C%3A_Speed_of_Sound (visited on 11/21/2025).

[6] Maxim Integrated Products. *DS18B20 Programmable Resolution 1-Wire Digital Thermometer*. 2008.

[7] HandsOn Technology. *User Guide: Ultrasonic Sensor V1.0*.

[8] Ben Wynne Yanyan Gao. *Data Acquisition and Handling DAH*. 2025.

[9] ztableblog. *Z Table*. URL: https://www.ztable.net/ (visited on 11/23/2025).

Appendix A Supplementary Material

- Python scripts used for the motion detector (sensor.py and detection.py)
- Python script used to create figures from the laboratory measurements (sensor.py and fig.py)
- Raw data in the form of .ods spreadsheets, .txt log files and .pdf figures
- README.txt explaining the supplementary material in more detail
- .pdf scan of the lab book used throughout the project
- .mp4 file of a video demonstration of real time motion detection